

Reflection

WHAT

During this module, I focused on structuring the codebase using principles aligned with distributed systems, including modular design, separation of concerns, and strict input validation. This resulted in a well-defined API with predictable behaviour and a clear internal structure. While the assignment did not explicitly require a distributed architecture, I deliberately approached it with scalability and robustness in mind.

Alongside the technical work, I participated in teamwork activities that exposed both strengths and weaknesses in my approach to collaboration. In practice, work allocation within the team was not always clearly defined. Tasks were often implicitly assumed rather than explicitly assigned, which led to overlap in some areas and gaps in others. For example, validation logic and error handling were implemented independently by multiple contributors, resulting in inconsistencies that required rework.

Team meetings were irregular and, when they did occur, lacked a clear structure or agenda. As a result, discussions were often reactive rather than goal-oriented. I contributed actively to technical discussions but did not take responsibility for structuring these interactions, which in hindsight was a missed opportunity to improve team efficiency.

A critical incident occurred during the tutor demonstration session. Although my implementation was complete and functional in my development environment (Windows), I did not adequately prepare for running the system on my macOS device. During the demonstration, differences in environment configuration caused unexpected behaviour. This disrupted the session and negatively affected the overall quality of the demonstration.

SO WHAT

This experience highlighted that software development is not only about building a technically correct solution but also about ensuring that it can be reliably executed, demonstrated, and maintained. From a design perspective, applying distributed systems principles proved beneficial. It enabled me to create a modular and testable solution, supporting the learning outcome of designing and developing software that meets a given specification and remains adaptable.

However, the more significant learning emerged from the collaboration and delivery aspects. The lack of clear task allocation and structured meetings directly impacted team performance. Importantly, this was not solely a team failure; it reflected my own tendency to remain focused on technical execution rather than actively shaping the team process. According to Belbin (2010), effective teams require individuals to adopt complementary roles, including coordination and organisation. In this case, I did not step into such a role, even though my experience would have allowed me to do so.

This revealed a key limitation in my professional behaviour: I operate most effectively in structured environments but do not always take responsibility for creating that structure when it is absent. Instead, I tend to adapt to inefficiencies, which ultimately reduces both my own performance and that of the team.

The demonstration failure further reinforced the importance of operational readiness. I assumed that a working implementation equated to a ready-to-deliver solution. This assumption was incorrect. In professional practice, software must be validated in the context in which it is executed, including differences in operating systems, dependencies, and runtime environments. As noted by Bass et al. (2012), system quality is not only determined by design but also by how well it performs under real operational conditions. These insights directly support the module learning outcomes around identifying and managing security and operational risks, and designing software that remains robust under real-world conditions.

On a personal level, this incident was frustrating. I was confident in the quality of my implementation, yet unable to demonstrate it effectively. The gap between what I

knew I had built and what I was able to show created a strong sense of discomfort, which made the need for better preparation particularly memorable. That experience highlighted the importance of preparation, not only technically but also in terms of presentation and delivery.

This also revealed a broader pattern in my approach: I tend to rely on implicit assumptions rather than making them explicit and verifiable. In the team context, this resulted in unclear ownership and duplicated work. In the demonstration context, it led to the incorrect assumption that the execution environment would behave consistently across systems. Interestingly, this contrasts with my technical implementation, where strict input validation was applied to eliminate ambiguity. This inconsistency between how I design systems and how I manage processes is a critical insight.

Finally, this demonstrates my ability to critically analyse development problems beyond their surface symptoms, identifying underlying causes related to assumptions, process gaps, and environment control.

NOW WHAT

Going forward, I will formalise my development process to include environment validation and demonstration readiness as mandatory stages. This includes testing across multiple operating systems where relevant, or using containerisation technologies such as Docker to ensure consistency across environments. This approach reduces operational uncertainty and aligns with best practices in modern software development.

In addition, I will treat demonstrations as critical deliverables rather than informal checkpoints. This means rehearsing the demonstration, validating the environment in advance, and ensuring that all dependencies are controlled. By doing so, I can ensure that the quality of the implementation is accurately reflected during evaluation.

From a teamwork perspective, I will adopt a more proactive and structured role by introducing explicit task ownership using issue-based tracking (e.g. GitHub Issues), where each task is assigned, documented, and validated upon completion. I will also structure team interactions through short, goal-oriented meetings with predefined outcomes, ensuring that discussions translate directly into actionable work. Where misalignment occurs, I will address it immediately rather than allowing assumptions to persist.

In future projects, I would consciously apply Belbin's team role framework by positioning myself not only as a "Plant" or "Specialist", but also incorporating aspects of the "Coordinator" and "Shaper" roles to ensure alignment and progress within the team. This represents a shift from a purely technical contributor to a more balanced role that combines technical expertise with team facilitation.

Finally, I will continue to apply distributed systems thinking as a default design approach. Even in smaller projects, this mindset supports scalability, maintainability, and robustness. By combining strong architectural principles with improved collaboration and operational discipline, I aim to deliver solutions that are not only technically sound but also professionally executed and reliable in real-world contexts. In doing so, I will be better positioned to act as an effective member of a virtual development team, as required by the module outcomes.

References

Belbin, R.M. (2010) *Team Roles at Work*. 2nd edn. Oxford: Butterworth-Heinemann.

Bass, L., Clements, P. and Kazman, R. (2012) *Software Architecture in Practice*. 3rd edn. Boston: Addison-Wesley.

Schön, D.A. (1983) *The Reflective Practitioner: How Professionals Think in Action*. New York: Basic Books.

Rolfe, G., Freshwater, D. and Jasper, M. (2001) *Critical Reflection in Nursing and the Helping Professions*. Basingstoke: Palgrave Macmillan.